AI-Assisted Conversion of R Statistical Packages to Python: A Methodological Framework and Implementation Plan for the `rpart` Case Study

Project Brief and Technical Specification for Graduate Student

Jun Li (Principal Investigator, PI; jun.li@nd.edu)

May 4, 2026

Contents

1 Preamble: Purpose and Scope of This Document

This document is the consolidated technical brief for a research and software engineering project that the PI has designed and intends to assign to a graduate student. It synthesizes an extended period of design work by the PI, including: the framing of the broader research vision, the systematic selection of an appropriate target R package for the case study, an analysis of the chosen package's structure and feasibility, and the construction of a generalizable methodology for AI-assisted R-to-Python statistical package conversion. The PI is the originator and senior author of the methodology described here. The student's role, defined in detail in Section 8, is to execute the rpart conversion under the PI's direction, while contributing to and refining the methodology and the operational artifacts (Claude Code Skills, fixture infrastructure, decision and difficulty logs) that the methodology produces.

This document covers:

- The motivation and vision for the broader research program (Section 2);
- The reasoning behind the choice of the `rpart` package as the case study (Section 3);
- A technical analysis of the `rpart` package (Section 4);
- A general methodological framework for converting an R package to Python with AI assistance (Section 5);
- The design of Claude Code Skills as the operational embodiment of the methodology (Section 6);
- The implementation plan, scope, and milestones for the `rpart` conversion (Section 7);
- The student's role, expectations, and authorship arrangements (Section 8);
- Required practices — difficulty logs, decision logs, and skill generality testing (Section 9);
- Publication and funding strategy (Section 10);
- Risks and mitigations (Section 11);
- Appendices, including the candidate-package comparison, `rpart` popularity statistics, and reference notes on PyPI naming policy (Section 12).

This is a working document. The student should treat it as authoritative direction from the PI for the project's intent and structure, and as a living reference to consult throughout the project.

2 Motivation and Vision

2.1 The R–Python Two-Language Problem in Statistics

The statistical computing ecosystem suffers from a persistent two-language divide. The R ecosystem contains thousands of high-quality statistical and machine-learning packages, many developed by domain experts—statisticians, biostatisticians, econometricians—who chose R because of its expressiveness for statistical modeling. The Python ecosystem has, over the past decade, become the dominant language for general data science, machine learning, and scientific computing more broadly. Python's growth is now sufficiently large that for many practitioners, the cost of context-switching

to R for an isolated method is high enough that they either reimplement the method poorly or work around it.

This divide is not symmetric. R's statistical strength outpaces Python's in several active areas: survival analysis, mixed-effects models, generalized additive models, quantile regression, recursive partitioning with surrogate splits, smoothing splines with total variation penalties, and others. Bridging tools such as `rpy2` and `reticulate` let users *call* across languages but do not produce native Python packages; they require both runtimes installed and impose performance, packaging, and debugging penalties that prevent serious adoption in production Python workflows.

The traditional response—"port it to Python"—has been undertaken sporadically by individual contributors, with notable failures: the Civis `python-glmnet` wrapper covers only a fraction of R's `glmnet`; `quantregpy` is essentially unmaintained; `py-earth` (the MARS port in `scikit-learn-contrib`) is abandoned and works only on Python 3.6 or earlier. The pattern is recognizable: a one-off effort produces a partial port, the original author loses interest or capacity, the package falls behind upstream R changes, and the Python community is left with a stale, incomplete substitute that does not earn user trust.

2.2 The Opportunity Created by Modern AI Tools

The economics of this kind of conversion work have changed materially in the last two years. Large language models with strong code-generation capabilities—accessed via Claude, GPT, and similar systems—can perform the bulk syntactic translation of R orchestration code to idiomatic Python in a small fraction of the time that hand-translation requires. They can also produce competent first-draft NumPy-style docstrings from R `.Rd` files, propose argument-mapping conventions across language idioms, and generate scaffolding for build systems and tests.

The bottleneck, however, is no longer code generation. The bottleneck is the *engineering methodology* surrounding the AI's output: how to verify correctness against the original R package, how to manage compiled code that the AI cannot translate safely, how to structure tests so that function-level fidelity is enforced, how to maintain the converted package as the upstream R package evolves, how to handle systematic R–Python semantic differences, and how to package and distribute the result for end users without imposing a compiler on them.

2.3 The Broader Research Vision

The PI's vision is therefore not the production of a single converted Python package, but the construction of a generalizable, reusable, *operationalized* methodology for AI-assisted R-to-Python statistical package conversion. The intended outcomes are:

1. A faithful Python conversion of one important R package, serving as a worked case study and as a usable artifact that demonstrates the methodology's viability.

2. A methodology paper, suitable for the Journal of Statistical Software or a comparable venue, that documents the framework in durable, language-agnostic form.

3. An executable operationalization of the methodology in the form of a suite of Claude Code Skills, allowing future practitioners to invoke the methodology directly when porting other R packages.

The student will lead the implementation of (1), contribute substantially to (3), and be a co-author on (2). The PI has originated the broader research vision, designed the initial methodology framework,

selected the case-study package, and is responsible for the strategic positioning of the work and the overall research direction. The methodology framework is expected to be refined and extended through the student's execution and through any proposals the student brings.

3 Selection of the Target Package: A Reasoned Journey

The choice of which R package to convert as the case study is a decision the PI made deliberately and iteratively, with substantial design effort and external evidence-gathering. This section documents the journey because the reasoning is itself part of the methodology—future practitioners should follow a similar process for selecting their own target packages, and the considerations below should be incorporated into the relevant Claude Code Skill. The student should understand the full reasoning, not merely the conclusion, because (a) it informs the framing of the methodology paper, and (b) it is the kind of judgment the student will need to apply later if the program scales to additional packages.

3.1 Initial Hypothesis: `glmnet`

The PI's initial candidate was `glmnet`, on the grounds of its extreme citation impact (Friedman, Hastie, Tibshirani et al. are titans of modern statistical learning) and the manifest weakness of `scikit-learn`'s elastic-net family relative to `glmnet`'s unified, regularization-path-aware, multi-family GLM solver. After investigating, however, two existing `glmnet` Python wrappers were identified (Civis Analytics and B.J. Balakumar, the latter having been developed in connection with Hastie). Both are Fortran wrappers, both lag the R version, and both require a Fortran compiler to install from source. More importantly, James Yang and Trevor Hastie have an in-progress C++ package called `adelie` that aims to provide group elastic net with Python bindings; this is from Hastie's own group and likely represents the future direction of `glmnet`-like functionality in Python. **Conclusion**: `glmnet` is too large to convert as a first demonstration, and the gap is being addressed by the original author's own group. *Rejected.*

3.2 Second Hypothesis: `quantreg`

The PI then considered `quantreg` (Roger Koenker), reasoning that quantile regression has clear methodological importance, the existing Python attempts (`quantregpy`) are abandoned, and Python's coverage of full quantile-regression inference is genuinely poor. An extended design conversation produced a complete plan for the conversion, including a layered architecture, fixture-based testing, the wrapping-vs-rewriting decision (the PI elected to wrap the original Fortran rather than reimplement it in pure Python, on grounds of correctness and performance), `cibuildwheel` for distribution, and an automated upstream-sync pipeline.

After this plan was assembled, the PI directly inspected the `quantreg` source and identified two decisive concerns:

1. **Dependency complexity.** `quantreg` imports `Matrix`, `MatrixModels`, `SparseM`, `survival`, and `MASS`. Each of these is a non-trivial R package in its own right. `SparseM` alone is a substantial sparse-linear-algebra package whose interface is used pervasively throughout the `rqss` and `sfn` components. Converting `quantreg` thus requires either depending on Python equivalents (`scipy.sparse`, `lifelines`) and translating every interface point, or partially converting the dependency packages themselves. Either path adds weeks of work that does not advance the methodology.

2. **The MVP problem.** `quantreg` is a sprawling toolkit with multiple solver families (`rq.fit.br`, `rq.fit.fnb`, `rq.fit.fnc`, `rq.fit.sfn`, nonlinear, censored, dynamic, total-variation-penalized splines, bootstrap inference, ANOVA, etc.). A complete conversion would consume 6–9 months of focused work. An MVP covering only `rq()` with two solvers would compete unfavorably with the existing `quantregpy` (which covers similar ground) and would not earn user trust, because users coming from R want complete functionality, not a partial substitute.

Conclusion: `quantreg` is too dependency-heavy and too sprawling for a clean case-study conversion. *Rejected.*

The PI considers this rejection an important methodological insight, and it should be encoded as guidance in the package-selection skill: **a strong first-conversion target should have minimal external R-package dependencies, ideally only `stats` and base R.**

3.3 Third Candidate: `earth` (MARS)

The PI then considered `earth` (Stephen Milborrow) implementing Friedman's MARS algorithm. `earth` satisfied several criteria: modest size, abandoned Python equivalent (`py-earth` is Python 3.6 only), C-based compiled code (easier wrapping than Fortran), and recognized algorithmic pedigree.

The PI raised two concerns that proved decisive on critical examination:

1. **Algorithmic context**: It is important to note (this was a point of factual confusion that was clarified) that the `gbm` package does *not* implement MARS; it implements gradient boosting machines, a different Friedman algorithm. The R MARS implementations are `earth` and the older `mda::mars`. So `earth` retains its canonical-implementation status.

2. **Method displacement**: MARS has been substantially displaced for predictive modeling on tabular data by gradient-boosted-tree methods (XGBoost, LightGBM, CatBoost). While MARS retains niches—interpretability, smaller-sample inference, certain applied domains—it is no longer where active demand sits. This means a faithful Python `earth` port would have a limited adoption ceiling, regardless of engineering quality, and the methodology paper would lack compelling user-demand evidence.

Conclusion: `earth` is technically tractable but lacks sufficient ongoing user demand to validate the methodology with strong adoption signals. *Rejected.*

This refinement of the selection criterion is itself a methodological contribution: **the target package should implement methods still in active use—particularly methods producing inferential output, exposing interpretable structure, or addressing applied-domain needs that modern black-box ML has not displaced.**

3.4 Final Choice: `rpart`

The PI selected `rpart` (Therneau & Atkinson, with Ripley's original R port) as the case study. The reasoning is detailed in Section 4, but the headline considerations are:

- **Zero external R-package dependencies.** `rpart` depends only on base R, `stats`, `graphics`, and `grDevices`. The dependency rabbit hole that sank `quantreg` is absent.
- **Compact R surface.** 2,198 lines of R across 36 files, most under 100 lines each.

- **Compact, well-bounded compiled-code surface.** 4,454 lines of pure C with only five `.Call()` entry points (`rpart`, `xpred`, `pred_rpart`, `rpartexp2`, `init_rpcallback`).
- **Recommended-package status.** `rpart` is one of the ∼30 packages bundled with R itself.
- **Active maintenance.** Version 4.1.27 was released in March 2026 (current at time of writing). The package is not moribund; an upstream-sync demonstration is meaningful.
- **Genuine, demonstrable Python gap.** `scikit-learn`'s `DecisionTreeClassifier/DecisionTreeRegress` lacks several `rpart` features that real users hit regularly: surrogate splits for missing values, the cross-validated cost-complexity (`cp`) table, survival trees (`method = "exp"`), Poisson trees (`method = "poisson"`), user-defined splitting functions via the C-callback mechanism, and per-variable splitting cost.
- **Heavy ecosystem entrenchment.** On CRAN, `rpart` has 30 reverse-depends, 119 reverse-imports, 146 reverse-suggests, and 5 reverse-enhances, totaling 300 packages. Hard dependents include the `caret`, `mlr3`, `tidymodels` ecosystems and major biostatistical packages (`Hmisc`, `rms`, `mice`, `riskRegression`).
- **Pre-existing test infrastructure.** 13 test scripts with accompanying `.Rout.save` reference outputs exist, substantially reducing the work required to bootstrap a fixture-based comparison framework.
- **Author pedigree.** Terry Therneau (Mayo Clinic, also author of the `survival` package) is one of the most respected biostatistical software authors active today.

The Python-port landscape was verified directly. The PyPI package called `rpart` (by Qiushi Yan) is unrelated to the R package: it is a small from-scratch decision-tree implementation, two releases on the same day in April 2023 with no subsequent activity, ∼38 weekly downloads, and contains only basic gini/entropy decision-tree and random-forest classes. It does not implement any of `rpart`'s distinctive features (surrogate splits, cp table, survival/Poisson trees, user-defined splits, formula interface). **No serious Python port of `rpart` exists.**

3.5 The Selection Process as a Methodological Artifact

The journey above—glmnet → quantreg → earth → rpart—is itself a contribution. It articulates and validates a set of selection criteria for AI-assisted conversion targets that did not exist in the prior literature. These criteria, in priority order, are:

1. Minimal external R-package dependencies (ideally only base R).
2. Compact, well-bounded compiled-code surface.
3. Real, demonstrable Python ecosystem gap (specific features users want and cannot get from existing Python packages).
4. Method still in active use; not displaced by modern alternatives.
5. Active upstream maintenance, so upstream-sync demonstration is meaningful.
6. Recognizable author pedigree, ideally with willingness to engage with the porting effort.
7. Pre-existing test infrastructure or vignettes that can seed the fixture framework.

8. Strong reverse-dependency footprint, indicating ecosystem entrenchment and a real user base for the converted package.

This list, refined further with the lessons from the rpart case study, should appear in the methodology paper and as the core content of the `r-package-recon` skill described in Section 6.

4 Technical Analysis of the `rpart` Package

This section is a structural analysis of `rpart` version 4.1.27 that the PI has performed on the source tree. The student should read this section in conjunction with the source itself; the goal is to prevent the student from re-deriving structural understanding the PI has already worked out.

4.1 Top-Level Composition

Component	Approximate size
R source files (`R/`)	2,198 lines across 36 files
C source files (`src/`)	4,454 lines across 25 `.c` + 4 `.h` files
Documentation (`man/`)	29 `.Rd` help files
Vignettes (`vignettes/`)	2 (`longintro.Rnw`, `usercode.Rnw`)
Tests (`tests/`)	13 `.R` scripts with `.Rout.save` reference outputs
Data (`data/`)	1 dataset (`stagec`)

License: GPL-2 | GPL-3. Authors: Terry Therneau (Mayo, *aut*), Beth Atkinson (Mayo, *aut, cre*), Brian Ripley (*trl*, original R port maintainer 1999–2017). Maintainer: Beth Atkinson.

4.2 Compiled-Code Interface

The C-to-R boundary uses R's modern `.Call()` interface (with `SEXP` arguments), not the older `.C()` or `.Fortran()` interfaces. This is good news for the Python wrapping work: `.Call()` makes the boundary explicit and the argument types are well-documented in `src/init.c`. The five entry points and their argument counts are:

Entry point	Purpose	# args
`rpart`	Build the tree (main entry)	11
`xpred`	Cross-validated predictions for cp grid	15
`pred_rpart`	Predict on new data	12
`rpartexp2`	Exponential transform for survival trees	2
`init_rpcallback`	Set up R-callback for user-defined splits	5

The Python wrapping strategy will use `pybind11` (preferred) or `cffi` (acceptable alternative) to bind these five functions. For the `init_rpcallback` entry point in particular, `pybind11`'s support for passing Python callables across the boundary as `std::function` or `py::function` is the cleanest available mechanism.

4.3 R Source Layout

The R sources cluster naturally into roughly four functional groups:

- **Main entry point and method-specific logic**: `rpart.R` (298 lines), `rpart.anova.R`, `rpart.class.R`, `rpart.poisson.R`, `rpart.exp.R`, `rpart.matrix.R`, `rpart.control.R`.

- **S3 methods on `rpart` objects**: `predict.rpart.R`, `print.rpart.R`, `summary.rpart.R`, `plot.rpart.R`, `text.rpart.R`, `residuals.rpart.R`, `labels.rpart.R`, `model.frame.rpart.R`, `prune.rpart.R`, `snip.rpart.R`, `na.rpart.R`, `post.rpart.R`, `meanvar.rpart.R`.
- **Cross-validation, complexity-parameter table, and tools**: `xpred.rpart.R` (143 lines), `plotcp.R`, `printcp.R`, `rsq.rpart.R`, `importance.R`, `path.rpart.R`, `rpartco.R`, `rpart.branch.R`, `rpartcallback.R`, `pred.rpart.R`, `roc.rpart.R`.
- **Utilities**: `formatg.R`, `prune.R`, `post.R`, `snip.rpart.mouse.R`, `zzz.R` (`.onLoad` / startup).

The S3 method system is registered in `NAMESPACE` with eleven `S3method(...)` declarations: `labels`, `meanvar`, `model.frame`, `plot`, `post`, `predict`, `print`, `prune`, `residuals`, `summary`, and `text`, all dispatched on class `"rpart"`.

4.4 Algorithmic Components in C

The 25 C files implement the actual CART algorithm. The student does *not* need to convert these, but does need to understand them well enough to wrap them correctly and to debug numerical discrepancies. Key files include:

- `rpart.c`, `rpart.h` — main entry and shared types
- `partition.c`, `bsplit.c` — recursive partitioning and best-split search
- `anova.c`, `gini.c`, `poisson.c` — per-method splitting criteria; `usersplit.c` for user-defined criteria
- `xpred.c`, `xval.c` — cross-validation machinery
- `surrogate.c`, `choose_surg.c`, `nodesplit.c` — surrogate-split logic
- `make_cp_table.c`, `make_cp_list.c`, `fix_cp.c` — cp-table construction
- `rpart_callback.c` — R-side callback mechanism for user splits; this is where Python-callable support must be threaded through during wrapping

4.5 Distinctive Features That Justify the Conversion

These are the features that distinguish `rpart` from `scikit-learn`'s tree implementations and constitute the adoption case for the Python port. Each must be preserved with fidelity in the conversion:

1. **Surrogate splits for missing values.** When the primary split variable is missing for an observation, `rpart` uses ranked surrogate variables (re-derived for each split) rather than imputing or excluding the observation. This is a Breiman-Friedman-Olshen-Stone original feature absent in `scikit-learn`.
2. **Cross-validated cp table.** A single `rpart()` call produces a complete table of cross-validated error across all meaningful complexity parameters, enabling 1-SE-rule pruning as a single workflow step. `scikit-learn` requires the user to construct this manually via `GridSearchCV` over `ccp_alpha`.
3. **Survival trees** (`method = "exp"`) using the Poisson-equivalent transform. `scikit-learn` has no survival trees; `scikit-survival`'s `SurvivalTree` uses a different algorithm.

4. **Poisson trees** (`method = "poisson"`) for count and rate data. Used in actuarial and epidemiological practice. Absent in `scikit-learn`.
5. **User-defined splitting functions** via the C-callback mechanism. Permits research and teaching uses where novel splitting criteria are explored. No equivalent in `scikit-learn`.
6. **Per-variable splitting cost** via the `cost` parameter, supporting cost-sensitive splitting beyond class-weights. Absent in `scikit-learn`.

These features—and not CART itself—are the value proposition the Python port must articulate clearly to its users.

5 General Methodological Framework

This section sets out the general methodology that the PI has designed for AI-assisted conversion of R packages to Python. It is presented here as guidance the student should follow during the rpart conversion; in the methodology paper it will be presented as the abstract framework the rpart case study validates.

The framework consists of nine phases. Phases are not strictly sequential; testing in particular runs continuously alongside conversion. But there is a logical ordering—you cannot convert a function without first understanding the package, and you cannot test a function without first generating reference fixtures.

5.1 Phase 1: Reconnaissance and Structural Analysis

Before a single line is written, the package's structure must be mapped. For `rpart`, much of this is captured in Section 4, but the student should re-validate the PI's analysis and extend it to function-call granularity. Specifically:

- Read `DESCRIPTION` for package metadata, dependencies, and license.
- Read `NAMESPACE` for the exported API surface and S3 method dispatch table.
- Identify all `.Call()`, `.C()`, `.Fortran()`, `useDynLib` entries to enumerate the compiled-code boundary.
- For each R file, produce a list of every function defined, every internal function it calls, and every compiled-code routine it invokes. This is feasible to do with AI assistance: feed the R source files to Claude and ask for the dependency graph. Cross-check against `NAMESPACE`'s `S3method()` entries to catch implicit dispatch relationships.
- Sort functions into layers from leaf (no internal dependencies) to top-level (entry points). Conversion proceeds bottom-up.
- Read all `.Rd` files and the vignettes. The vignettes in particular contain narrative explanations and worked examples that illuminate intent in ways the source code does not.

The output of Phase 1 is a structured analysis document checked into the project repository: `docs/architecture-analysis.md`. This document is also the input to the `r-package-recon` Claude Code Skill (Section 6).

5.2 Phase 2: Wrapping Compiled Code

For `rpart`'s C source, the wrapping strategy is:

- Use `pybind11` (preferred) for binding generation. Rationale: clean interface, excellent NumPy interoperability, first-class support for passing Python callables across the boundary (needed for the user-defined split mechanism). `cffi` is an acceptable alternative if `pybind11` proves problematic on a specific platform.
- Use `Meson` as the build system, with `meson-python` as the PEP 517 backend. Rationale: `Meson` is now the standard build system for scientific Python packages with compiled extensions (NumPy and SciPy moved to it).
- Use `cibuildwheel` in GitHub Actions to build pre-compiled wheels for Linux x86_64, macOS x86_64, macOS arm64, and Windows x86_64, across Python versions 3.10 through 3.13. Wheels are uploaded to PyPI on tagged releases.
- The first compiled-code task should be wrapping `rpartexp2` (only 2 arguments, only 51 lines of C). This is a calibration exercise to validate the toolchain end-to-end before tackling the larger entry points.

The user-defined-split callback (`init_rpcallback`) is the trickiest wrapping task, because it requires the C tree-growing code to call back into Python during execution. The strategy is to register a `pybind11`-managed Python callable as the callback, with careful attention to the GIL (acquired before each callback) and to any persistent state the C code expects to find.

5.3 Phase 3: Building the Reference Fixture Framework

This is the heart of the correctness story. The methodology relies on running every converted function against reference outputs generated by the original R package, with comparisons made within specified numerical tolerances.

Three test strategies, used in combination:

1. **Direct call.** For functions whose inputs are plain numerical objects (matrices, vectors, scalars), call them directly from R, save inputs and outputs to JSON or RDS, and compare in Python. Most leaf-layer functions and many Phase 2 wrapped C functions fit here.
2. **Input capture via instrumentation.** For internal functions whose inputs are intermediate objects produced deeper inside the R call stack, monkey-patch the function in R to save its arguments and return value to disk during a normal end-to-end call. The Python test then loads the captured input and compares the Python function's output to the captured R output. This is essential for functions operating on, e.g., the partially-built tree object during recursive partitioning.
3. **End-to-end.** Run an entire `rpart()` call in both R and Python on the same dataset, compare top-level outputs (tree structure, cp table, predictions). Catches compositional errors that function-level tests miss.

The student will write a single comprehensive R script, `generate_test_fixtures.R`, that runs a battery of `rpart` calls (varying datasets, methods, control parameters, missing-value patterns)

and dumps inputs and outputs at every relevant layer to `tests/reference/`. This script is re-run whenever the upstream R `rpart` version is updated.

The 13 existing test scripts with `.Rout.save` files in the upstream package are a substantial head start. They should be incorporated as additional reference cases.

Tolerance policy. Comparisons must specify a numerical tolerance. The default for `rpart` should be relative tolerance of 10^{-8} for floating-point outputs and exact equality for integer/categorical outputs (split variables, node counts, class predictions). When a discrepancy exceeds tolerance, it is logged in the difficulty log (Section 9) and triaged: it is either a conversion bug, an upstream R bug, or a floating-point reordering. All three categories must be documented; the third category is acceptable but not silent.

5.4 Phase 4: Conversion of the R Orchestration Layer

This is the phase where AI assistance is most valuable. The R orchestration layer in `rpart`—formula handling, model frame construction, dispatch to C, post-processing of returned tree objects, S3 method bodies—is well within the range of high-quality first-draft AI translation. The student's role is the verification loop: (i) feed an R function to Claude, (ii) review the proposed Python translation for correctness and idiom, (iii) run the corresponding fixture-based test, (iv) debug any discrepancies, (v) commit and document.

The conversion order is bottom-up: convert leaf functions first (those with no internal dependencies in the R code), then functions that call only already-converted functions, and so on up to the top-level entry points. This ordering is enforced by the dependency graph from Phase 1.

5.5 Phase 5: Handling R–Python Semantic Differences

Several systematic differences between R and Python require deliberate design decisions. These should be made once, recorded in the decision log, and applied uniformly across the package.

- **Formula interface.** `rpart`'s primary API is formula-based. The Python port should support both formula-style (via `patsy` or `formulaic`) and matrix-style (`X`, `y`) calls. Recommended primary API is matrix-based with formula as a convenience layer for R users transitioning.
- **Indexing offset.** R is 1-indexed; Python is 0-indexed. Wherever indices cross the C boundary or are returned to the user (e.g., split variable indices), the offset must be applied consistently. Every such crossing should be flagged in the code with a short comment.
- **Missing values.** R's `NA` maps to `np.nan` for floats; for integer and boolean arrays the situation is more complex (`pandas.NA` or sentinel values). The package should accept `np.nan` and route to `rpart`'s native missing-value handling (which is the whole point of the surrogate-split feature).
- **Return objects.** R's `rpart` returns a named list of class `"rpart"`. The Python equivalent is a proper class (e.g., `RpartTree`) with attributes (`frame`, `splits`, `cptable`, `variable.importance`, etc.) and methods (`predict`, `prune`, `summary`). Implement `__repr__`, `__str__`, and `_repr_html_` for Jupyter rendering.
- **scikit-learn API compatibility.** A scikit-learn-compatible wrapper class (`fit`/`predict`/`score`, inheriting from `BaseEstimator` and the appropriate `*Mixin`) should be provided as a secondary

API. This unlocks `Pipeline`, `GridSearchCV`, etc., for free, and is a substantive value-add over R.

- **Categorical/factor handling.** Accept `pandas.Categorical` and pre-coded numeric matrices. Provide a utility for category encoding consistent with R's default treatment-coding.
- **Print and summary formatting.** `rpart`'s `print()` and `summary()` produce specific formatted text outputs that R users recognize. The Python port should reproduce these textually (perhaps with a modest typographic upgrade), as well as providing DataFrame-returning equivalents (e.g., `tree.cptable_df()`).

5.6 Phase 6: Documentation Conversion

Each `.Rd` file in `man/` converts to a NumPy-style docstring on the corresponding Python function. The mapping is: `\description` → opening paragraph; `\arguments` → `Parameters` section; `\value` → `Returns` section; `\examples` → `Examples` section, with R code translated to Python; `\references` → `References` section. AI assistance is highly effective here, but the student must review every example to ensure it actually runs in Python.

The two vignettes (`longintro.Rnw`, `usercode.Rnw`) should be translated to Jupyter notebooks or Sphinx tutorials. These are the main onboarding materials for new users.

A Sphinx documentation site, hosted on Read the Docs, autodocs from the docstrings and includes the vignettes and an R-to-Python migration guide. The migration guide is a side-by-side translation of common R `rpart` idioms to their Python-package equivalents and is high-leverage for adoption.

Every docstring must credit the original R package and its authors. A standard line such as "This function is a Python port of the `rpart` R package by Therneau and Atkinson" should appear in each user-facing docstring.

5.7 Phase 7: Distribution

The distribution stack:

- **GitHub** for source code, issues, CI/CD.
- **PyPI** for end-user installation via `pip`, with pre-built wheels via `cibuildwheel`.
- **conda-forge** as a secondary channel for users in scientific Python or mixed R/Python environments.
- **Read the Docs** for documentation hosting.
- **Zenodo** for an archival DOI on each release.

PyPI naming: the existing `rpart` name on PyPI is held by an unrelated abandoned package (Section 12.3). The default plan is to launch under `pyrpart` and pursue PEP 541 abandonment-transfer of the `rpart` name in parallel without making it a blocker.

5.8 Phase 8: Upstream Synchronization

The package will go stale rapidly without an automated mechanism for tracking upstream R `rpart` updates. The methodology specifies:

- A scheduled GitHub Action that polls CRAN (or the `bethatkinson/rpart` GitHub mirror) weekly.
- When a new upstream version is detected, the action diffs the new R and C sources against the stored reference copy and classifies changes by file-type category: C-source changes (highest priority, may require re-wrapping), R-source changes (require re-conversion of affected functions), documentation-only changes (low priority), test changes (a gift—added to the fixture set), new files (require triage).
- A `mapping.yaml` file maintained alongside the source records the correspondence between R files/functions and Python files/functions, with linked Fortran/C dependencies. The diff classifier uses this to produce a focused list of Python files to re-examine.
- The classified diff is posted as an automatically-opened GitHub issue with a structured change report. A maintainer then re-converts the changed R functions (with AI assistance, using the diff as context), re-runs the fixture suite, and merges.

5.9 Phase 9: Discrepancy Auditing and Performance Optimization

After functional parity is achieved, a final auditing phase exercises the package on edge cases (degenerate inputs, near-singular cases, extreme values, large datasets, ill-conditioned problems) and catalogs every numerical discrepancy. Each discrepancy is classified as: (a) conversion bug (must fix), (b) upstream R bug (report to Therneau/Atkinson), or (c) floating-point reordering (document, do not fix).

Performance optimization, if pursued, focuses on the Python orchestration layer where AI tools can identify vectorization opportunities, redundant copies, and parallelization opportunities (cross-validation in particular is embarrassingly parallel and a strong candidate for `joblib`). The C code itself is not modified.

6 Operationalization via Claude Code Skills

A central methodological commitment of this project is that the methodology itself must be *operationalized*—rendered in a form that future practitioners can directly invoke, not merely read. The chosen vehicle is the Claude Code Skill: a structured instruction file that the Claude Code agent loads and follows when invoked, capturing methodological guidance as actionable, AI-executable directives.

6.1 Why This Operationalization Matters

Methodology papers in software engineering have a known weakness: they are cited but rarely *used*. Future practitioners read the paper, agree with it, and then proceed with their own ad hoc process. The lessons remain locked in the publication.

A Claude Code Skill changes this dynamic. The methodology becomes executable: a future practitioner porting, say, `KernSmooth` invokes the skill suite and is actively guided through the process—reconnaissance, planning, wrapping, fixture generation, discrepancy debugging—following the same engineering discipline the rpart conversion validated. The methodology is no longer documentation; it is infrastructure.

6.2 The Planned Skill Suite

The methodology decomposes into eight skills, each invocable independently and composable into a pipeline. Each skill has explicit inputs and outputs, audience documentation, and testable behavior.

1. `r-package-recon` — given a path to an R package source tree, produces a structured architecture analysis: dependencies, S3 dispatch table, compiled-code entry points, function-level call graph, layered conversion order. Output: `architecture-analysis.md` + `mapping.yaml` skeleton.

2. `r-to-python-conversion-planner` — given a recon output, produces a phased conversion plan with effort estimates, risk callouts, and milestone definitions.

3. `compiled-code-wrapper` — given the compiled-code boundary identified in recon, guides the wrapping work: choice between `pybind11`/`cffi`/`f2py` based on language and complexity, build-system scaffolding with Meson and `cibuildwheel`, calibration exercise on the simplest entry point first.

4. `r-fixture-generator` — generates `generate_test_fixtures.R` and the corresponding Python comparison harness, parameterized by the function list from the recon output.

5. `r-orchestration-converter` — guides the function-by-function conversion of R orchestration code to Python, with attention to S3 dispatch, formula handling, NA semantics, and indexing offsets.

6. `discrepancy-debugger` — protocol for diagnosing R-vs-Python output differences: triage as floating-point reordering / indexing / algorithmic / upstream bug, with decision tree.

7. `rd-to-numpy-docstring` — converts `.Rd` files to NumPy-style Python docstrings, including translation of R example code to Python.

8. `upstream-sync-setup` — generates the GitHub Actions workflow, change classifier, and `mapping.yaml` update logic for automated upstream tracking.

6.3 How the Skills Will Be Built

Three principles, each non-negotiable:

1. **Skills are written during the work, not after.** Memory fades; nuance is lost; the most valuable observations are the ones made in the moment of solving the problem. Every two to three weeks, immediately after completing a phase or substantial sub-phase, the student drafts the corresponding skill from the difficulty log (Section 9) and the freshly-experienced decisions.

2. **Skills are tested for generality.** The natural failure mode is to write skills that read as "how I did rpart," which are useless for future practitioners. After each skill draft, the student must dry-run it on a different small R package (candidates: `KernSmooth`, `acepack`, `logspline`) to verify the guidance still makes sense. If a skill contains rpart-specific assumptions, generalize them or explicitly mark them as case-study illustrations.

3. **Skills are written as instructions to a smart assistant, not as documentation for human readers.** The register, structure, and information density should be tuned for AI

execution: explicit inputs and outputs, explicit decision rules, explicit failure modes, no prose padding. Validate by actually invoking each skill on test cases.

6.4 Risks Specific to Tool-Operationalized Methodology

- **Tool-version dependency.** Claude Code Skills are an Anthropic product currently under active development. The skill format may evolve. Mitigation: the methodology paper documents the methodology in language-agnostic prose, so that the durable intellectual content survives any change in the operational format. Skills are "current best operationalization," not the methodology itself.
- **Reproducibility for academic review.** Reviewers without Claude Code access must still be able to evaluate the methodology. Mitigation: the paper includes enough text-form description of each skill that the methodology is fully accessible without running anything; the skills are supplementary material.
- **Trendiness perception.** Reviewers may dismiss "we used AI tools" framing as faddish. Mitigation: the paper leads with engineering methodology—verification, dependency analysis, upstream sync—and presents skills as the operational embodiment, not the headline.
- **Model-specific behavior.** Different Claude versions may behave differently. Mitigation: test each skill on multiple model versions during development; document any model-specific behaviors as known limitations.

7 Implementation Plan, Scope, and Milestones

7.1 Phased Scope Definition

The PI defines three phases of completeness. The student should treat these as a contract: "done" means meeting the explicit criteria for the relevant phase.

Phase A — Minimum Viable Port (target: weeks 1–6 of student work).

- Build system: Meson + `pybind11` + `cibuildwheel`; wheels building successfully for Linux, macOS x86_64, macOS arm64, Windows on Python 3.11/3.12.
- Wrapping: `rpart`, `pred_rpart`, `rpartexp2` entry points wrapped and callable from Python.
- Methods: `method = "anova"` and `method = "class"` only.
- Functionality: `rpart()` fitting, `predict.rpart`, `print.rpart` (text representation), basic `summary.rpart`, `prune.rpart`.
- Fixture framework: in place with at least 20 reference cases across the supported methods.
- Documentation: docstrings for the public API; brief README; installation instructions.
- Distribution: package installable from a TestPyPI account via `pip`.

Phase B — Full Functional Parity (target: weeks 7–16, ending around month 4).

- All four methods: `anova`, `class`, `poisson`, `exp`.

- All five C entry points wrapped, including `xpred` and `init_rpcallback`.
- Surrogate splits fully functional.
- Cross-validation cp table (`xpred.rpart` + `plotcp`/`printcp`) functional.
- User-defined splitting functions via Python callable.
- All eleven S3 methods implemented as Python equivalents.
- All 29 `.Rd` files converted to NumPy-style docstrings.
- Both vignettes translated to Jupyter notebooks.
- Fixture suite covers all upstream `.Rout.save` cases plus additional edge cases.
- Public release on PyPI under a stable name.

Phase C — Polish and Methodology Artifacts (target: weeks 17–20, ending around month 5).

- conda-forge recipe submitted and accepted.
- Read the Docs site published; R-to-Python migration guide written.
- Upstream-sync GitHub Actions workflow operational.
- All eight Claude Code Skills drafted, tested, and published.
- JOSS submission for the software paper prepared.
- Decision log and difficulty log finalized as supplementary material.
- Initial draft of the methodology section of the JSS paper.

7.2 Sequencing Guidance

The first two weeks of the student's work should be infrastructure-only. This is non-negotiable. The student should expect:

- Week 1: Set up the project repository, virtual environments, directory skeleton, `pyproject.toml` with `meson-python` backend, basic Meson `meson.build`, and a successful local build of a minimal hello-world C extension via `pybind11`.
- Week 2: Wrap `rpartexp2` (the simplest C entry point) with passing fixture-based tests, and stand up an initial `cibuildwheel` configuration in GitHub Actions producing wheels on at least Linux and macOS. Begin the `generate_test_fixtures.R` skeleton. This is the calibration exercise—if Week 2 stretches significantly longer, the issue is almost certainly toolchain-related and should be diagnosed before proceeding.

The student should be told explicitly: *the first two weeks are infrastructure, not algorithm.* This is the most frustrating phase; it will feel like no progress is being made. This is normal and expected. Windows wheels and macOS arm64 may need additional iteration during the first half of Phase A; this is treated as expected calendar overhead, not as schedule slip.

7.3 Realistic Timeline Caveats

The PI's timeline estimate (Phase A in weeks 1–6, B in weeks 7–16, C in weeks 17–20) is approximately 4.5 months total, with an outer bound of 5 months. It assumes:

- A focused, full-time effort by a strong student with a solid programming background, including prior comfort with build tooling, version control, and Python packaging.
- AI assistance throughout the orchestration-conversion phase.
- Active weekly engagement by the PI for design and debugging decisions.
- No major personal or institutional disruptions to the student's work schedule.

The compressible parts of this schedule are infrastructure setup (strong programmers move through it faster), documentation conversion (AI-assisted `.Rd`-to-docstring is genuinely fast), and the S3-method porting (most methods are short). The non-compressible parts—and where calendar time will be consumed regardless of student strength—are cross-platform wheel builds, the `init_rpcallback` Python-callable mechanism (genuinely difficult; budget 1.5–2 weeks specifically for it, or defer to a later release), and discrepancy debugging (mismatches between R and Python outputs surface at unpredictable rates and cannot be rushed without compromising correctness, which is the entire point of the methodology).

If any of the assumptions above is violated, the timeline extends. The student should not be penalized for honest schedule slip; what matters is that slippage is communicated early so that scope (not quality) is what gives.

8 The Student's Role and Authorship

8.1 What the Student Does

The student is the implementer of the rpart case study and a contributor to the methodology operationalization. Specifically:

- Executes Phases 1–9 of the methodology on the rpart package, producing the working Python package.
- Maintains the difficulty log and decision log throughout.
- Drafts the eight Claude Code Skills, refining them as the rpart conversion produces evidence about their content.
- Contributes substantively to the JOSS software paper, likely as first author given that they are the implementer.
- Contributes to the methodology paper as co-author, primarily through the case-study sections and through evidence gathered in the difficulty log.
- Engages with upstream maintainers (Therneau, Atkinson) on questions arising during conversion; this should be coordinated with the PI to ensure professional tone and appropriate framing.

8.2 What the PI Does

The PI is the senior author of the methodology and the primary intellectual contributor to its design. Specifically:

- Originated the broader research vision (the AI-assisted-R-to-Python program).
- Designed and articulated the methodology framework described in Sections 5–6.
- Selected the rpart package as the case study, after considering and rejecting glmnet, quantreg, and earth on the reasoned grounds documented in Section 3.
- Performed the initial structural analysis of rpart and of the rejected candidates.
- Verified the Python-port landscape (no serious existing port) and gathered the upstream popularity statistics that justify the choice.
- Designed the publication and funding strategy (Section 10).
- Will provide weekly engagement during the conversion to guide design decisions, debug discrepancies, review the difficulty and decision logs, and ensure the methodology insights are captured and articulated.
- Will be first author on the methodology paper (JSS or comparable venue).
- Will be senior/corresponding author on the software paper (JOSS), with the student likely as first author.

8.3 Authorship Defaults

The default authorship arrangements, agreed in advance and recorded here:

- **Methodology paper (JSS or equivalent)**: PI first author, student second author.
- **Software paper (JOSS)**: Student first author, PI senior author.
- **Package metadata**: PI as principal author, student as co-author and current maintainer; original R authors (Therneau, Atkinson, Ripley) credited prominently as "original R package authors."
- **Claude Code Skills**: PI and student joint authorship; skill files include both names in their metadata.

These defaults are revisitable if circumstances meaningfully change (e.g., the student takes on substantial additional methodology contributions beyond the scope above), but should not be renegotiated casually mid-project.

8.4 What This Project Is Not

It is important to be candid with the student about what this project is and is not, so that expectations align:

- This is fundamentally an engineering project rather than a theorem-proving research project: the core deliverable is a faithful Python conversion, not a new statistical method. The methodology framework reflects the PI's design work to date, but the student's execution

is expected to refine and extend it — every difficulty resolved, every skill drafted, every discrepancy debugged contributes intellectual content. The student is also encouraged to propose new ideas at any point: methodological refinements, alternative target packages, or related research directions arising from the case study. Such contributions are welcome and will be reflected in authorship and credit.

- The student's research-trajectory benefit comes from the co-authored methodology paper, the lead-authored software paper, the demonstrable software-engineering accomplishment on a recognized package, and exposure to the broader research program (which may extend into the student's own thesis if their interests align).
- The student should approach this with the engineering discipline of a serious software developer. Sloppy work will not be tolerated, because the methodology paper's credibility depends on the case study's quality.

9 Required Practices: Logs and Generality Testing

This section sets out three required practices that are not optional. They are the difference between a project that produces a package and a project that produces methodology.

9.1 The Difficulty Log

A difficulty log is a structured record of every nontrivial obstacle encountered during the conversion, the diagnostic process, the root cause, and the resolution. It is the raw material for the methodology paper, the skills, and the credibility of the entire project.

Format. A single markdown file checked into the repository at `docs/difficulty-log.md`. Each entry has the following fields:

- **Date and phase**: when, and which methodology phase.
- **Symptom**: what went wrong, with the actual error message or numerical discrepancy. Include enough context to reconstruct the situation cold.
- **Hypothesis path**: what the student initially thought the cause was, what they tried, what they ruled out.
- **Root cause**: what the actual cause was. Distinguished from the symptom.
- **Resolution**: what fixed it. Include the general principle if any.
- **Time spent**: rough hours, for future estimation.
- **Generalizable lesson**: a one-sentence statement of what should go into a skill or the methodology paper. If there is no generalizable lesson, that itself is worth recording.

Discipline. Entries are made the same day the difficulty is resolved. Not later. The PI will check the log weekly. A week with active work and no log entries is a problem to be diagnosed immediately, not three months later.

Examples of what to log.

- Numerical discrepancies between R and Python outputs and their root causes.
- Build-system failures and their resolutions, especially cross-platform.
- Cases where the AI-generated translation was subtly wrong and how it was caught.
- False starts and abandoned approaches, with reasons.
- Unexpected ease—things that you expected to be hard but turned out simple, with the reason. (These are methodologically valuable too: future practitioners should not waste time preparing for difficulty that is not there.)

The log is intended to be *public*. It will be in the GitHub repository. A clean, professional log of well-described difficulties is credibility-building, not embarrassing. The student should approach each entry as something a future researcher will read and learn from.

9.2 The Decision Log

The decision log is separate from the difficulty log. Decisions are choices made *without* hitting a wall: design decisions, scope decisions, naming decisions, dependency-selection decisions, API decisions.

Format. A markdown file at `docs/decision-log.md`, in the form of an architecture decision records (ADR) collection. Each entry: title, status (proposed/accepted/superseded), context, decision, consequences, alternatives considered.

Examples.

- "Use `pybind11` for C bindings" — with the alternatives (`cffi`, `ctypes`, raw CPython API) explicitly listed and rejected with reasons.
- "Primary API is matrix-based; formula API is convenience layer" — with rationale.
- "cp table is returned as a structured object, not a raw matrix; structured object exposes a `.to_dataframe()` method" — with rationale.
- "Categorical variables: accept `pandas.Categorical` and pre-coded numeric matrices; do not accept raw `str` columns" — with rationale.

The decision log feeds the methodology paper's design rationale sections and the skills' decision rules.

9.3 Skill Generality Testing

Each Claude Code Skill, after being drafted from the rpart-conversion experience, must be validated for generality by dry-running it on a different R package. This is the single most important practice for ensuring the methodology contribution is genuine and not rpart-specific.

Procedure. For each skill drafted:

1. Identify a different small R package (`KernSmooth`, `acepack`, `logspline`, `MASS::lqs`, etc.).
2. Invoke the skill on this package's source.

3. Read the output critically. Does the guidance still make sense? Does it apply cleanly, or does it reveal rpart-specific assumptions buried in the wording?
4. If rpart-specific assumptions surface, generalize the skill or explicitly mark them as case-study illustrations.
5. Record the test in `docs/skill-generality-tests.md`.

This is not a full conversion—it is a few-hours validation. The student is not committing to converting these other packages. They are using them as test substrates for the methodology's generality.

10 Publication and Funding Strategy

10.1 Sequencing Principle

The PI's strategic decision, after extensive consideration, is to **build first, publish everything together**. No preprints, no GitHub teasers, no conference announcements until a working package exists and the methodology paper is in draft. The reasoning is threefold:

1. The methodology, while sound, is not so secret that a competent group could not arrive at something similar independently. Pre-announcement leaks the idea without protecting it.
2. A methodology paper without a working case study is much weaker than one with. The case study *is* the evidence that the methodology works.
3. Releasing the package and paper together gives nobody a window to scoop either piece.

10.2 Target Venues

Primary publication. Journal of Statistical Software (JSS), single combined paper: methodology framework + rpart case study, ∼30–50 pages. Rationale: JSS is the highest-impact venue in statistical computing, routinely publishes long-form papers combining methodology with deep software case studies, and explicitly accepts "computational infrastructure upon which implementations can be built."

Software paper. Journal of Open Source Software (JOSS), short (∼2–4 pages) paper for the rpart-py package itself. Low-overhead, fast review, provides a citable DOI. Submit when the package is stable and documented.

Future methodology paper. A second methodology paper distilling lessons from additional case studies as the program scales. This will be the durable, widely-cited methodology reference, much stronger than a single-case-study methodology paper could be. The current JSS paper establishes priority and demonstrates feasibility; the future paper consolidates and generalizes.

10.3 Engagement with Upstream

The PI will email Beth Atkinson and Terry Therneau early in the project—ideally within the first month—introducing the project, the student, and the goals. The framing is professional courtesy and optional collaboration: their endorsement, even tacit, would strengthen the project; their

guidance on subtle algorithmic choices would save the student weeks of confusion. Therneau and Atkinson are collegial maintainers; a hostile reaction is highly unlikely given the nature of the work.

11 Risks and Mitigations

Risk	Mitigation
Build-system pain (especially Windows wheels) consumes more time than expected.	First three weeks dedicated to infrastructure; Windows is treated as the calibration platform; `cibuildwheel` working from Week 3, not later.
The user-defined-split callback proves harder than expected to wrap.	It is acceptable to defer this feature to Phase B late or even Phase C; rpart-py without user-splits is still a usable Phase-A release.
Numerical discrepancies between R and Python outputs are pervasive and hard to triage.	The discrepancy-debugger skill embodies the diagnostic protocol; the difficulty log records every case; tolerance policy is set in advance; PI engagement is high during this phase.
The student's logs go stale and the methodology evidence is lost.	PI reads the logs weekly; a week with active work and no entries is treated as a process failure to be addressed at the next meeting, not at end-of-project.
Skills end up rpart-specific and not actually reusable.	Generality testing (Section 9) on at least one alternate package per skill, before publication.
Upstream changes its R sources during the conversion, invalidating fixtures.	Pin to a specific upstream version (4.1.27 as of project start). The upstream-sync mechanism handles future changes after release; during initial conversion, treat upstream as frozen.
Someone else publishes a similar Python port during the project.	Build first, publish together (Section 10); the methodology paper does not require uniqueness of the case-study port to make its contribution. If this happens, the methodology paper still stands and the case study can cite the parallel work.
Reviewer dismisses the AI-tools framing as faddish.	Lead the paper with engineering methodology, not AI-tools claims; skills are presented as the operational embodiment, not the headline.
Therneau or Atkinson objects to the project.	Engage them early, with a professional and respectful framing. The GPL license does not require their permission, but their goodwill is worth real effort.
Student finds the work too engineering-heavy and disengages.	The student must be selected with this dimension explicitly in mind (Section 8). A 1-week paid trial wrapping a single C function is a reasonable initial filter.
PyPI `rpart` name remains held.	Default plan is `pyrpart` as the package name; PEP 541 transfer is pursued in parallel without becoming a blocker.

Risk	Mitigation
GPL license deters some users (especially commercial).	Acceptable for a research/methodology package. The methodology paper notes this trade-off. The GPL is required because rpart's R source is GPL.

12 Appendices

12.1 Comparison Table: Candidate R Packages Considered

Package	R lines	Compiled lines	Deps	Entry pts	Status
`glmnet`	—	—	several	many	Rejected (existing wrappers; `adelie`)
`quantreg`	6,150	12,111 (Fortran)	5	24	Rejected (deps; sprawl)
`earth`	moderate	moderate (C)	few	few	Rejected (method displaced)
`rpart`	2,198	4,454 (C)	**0**	**5**	**Selected**

12.2 rpart Popularity Statistics (May 2026)

Captured for reference and for use in the JSS paper's motivation section.

- Last 7 days: ∼36,000 downloads (RStudio CRAN mirror).
- Last 30 days: ∼176,000 downloads.
- Last 12 months: ∼1,005,000 downloads (single-mirror; true cross-mirror total likely 2–3× this).
- Reverse depends on CRAN: 30 packages.
- Reverse imports on CRAN: 119 packages.
- Reverse suggests on CRAN: 146 packages.
- Reverse enhances on CRAN: 5 packages.
- Total reverse-dependency count: 300 packages.
- Recommended-package status (bundled with R itself).
- Notable hard dependents: `caret`, `mlr3` ecosystem (∼20 sub-packages), `parsnip`, `baguette`, `recipes`, `ipred`, `partykit`, `mice`, `Hmisc`, `rms`, `riskRegression`, `pec`, `SuperLearner`.
- Latest upstream version: 4.1.27, released March 26, 2026.

12.3 The Existing PyPI `rpart` Package

For the record: the PyPI name `rpart` is held by an unrelated package by Qiushi Yan, two releases on the same day (April 25, 2023), no subsequent activity, ∼38 weekly downloads. The package contains a small from-scratch `DecisionTreeClassifier` and `RandomForestClassifier` using gini/entropy and ChiMerge discretization, plus an accidentally-bundled `DecisionTree-trythis/` subdirectory. It does not implement any of the R `rpart`'s distinctive features. It is functionally abandoned but technically holds the name.

PEP 541. Python Enhancement Proposal 541 governs PyPI package-name retention and transfer. It defines three categories of transfer requests: abandoned projects (relevant here), invalid projects, and active disputes. For abandoned projects, the request process requires (a) good-faith contact attempts with the current maintainer, (b) demonstration that the package is actually abandoned, and (c) a legitimate use for the name. PEP 541 transfers typically take weeks to months, and the bar is conservative.

Plan. Launch under `pyrpart` as the immediate name. Concurrently, email Qiushi Yan via the listed Gmail address asking about a name transfer. If unresponsive after four to six weeks, file a PEP 541 transfer request via the `pypi/support` GitHub repository. Even if the `rpart` name is never acquired, `pyrpart` is a perfectly acceptable permanent name (cf. `pymc`, `pytables`, `pymongo`).

12.4 Suggested Initial Reading List for the Student

Before writing any code, the student should read or work through:

1. Therneau & Atkinson, "An Introduction to Recursive Partitioning Using the rpart Routines" (the `longintro` vignette in the rpart source).
2. Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, Chapter 9 (CART).
3. The full source of `R/rpart.R`, `R/predict.rpart.R`, `R/rpart.matrix.R`, and `R/xpred.rpart.R`, with cross-reference to `src/rpart.c` and `src/xpred.c`.
4. This document, in full, including this reading list.
5. The `pybind11` documentation, focusing on: function bindings, NumPy interoperability, and passing `py::function` as callbacks.
6. The `cibuildwheel` documentation.
7. PEP 517 / PEP 518 and the `meson-python` backend documentation.

After reading, the student should be able to articulate, in their own words: (i) why rpart exists when scikit-learn does, (ii) what the cp table is and why it matters, (iii) what surrogate splits are and why they are absent in scikit-learn, (iv) what the conversion methodology is and what artifacts it produces, (v) what their authorship arrangements are, and (vi) what the difficulty log is and why it matters. The PI will check these understandings in the kickoff meeting before any code is written.

12.5 First Week Concrete Deliverables

By end of Week 1 of the student's work on the project:

- Project repository created (GitHub, private initially); directory structure established.
- `pyproject.toml` with `meson-python` backend.
- Minimal hello-world C extension built and importable from Python.
- First entry in `docs/decision-log.md`: "Use `pybind11` for C bindings" (with alternatives considered).
- First entry in `docs/difficulty-log.md` (likely a toolchain-related entry from the hello-world build).

- `docs/architecture-analysis.md` drafted from Section 4 of this document, extended with the student's own function-level reading.
- Kickoff meeting with PI, in which the student demonstrates the hello-world build and articulates their reading-list understanding.

Closing Note

This document represents a substantial investment of design effort by the PI: the broader research vision, the systematic candidate-package selection, the technical analysis of rpart, the methodology framework, the publication and funding strategy, and the operationalization-via-skills approach are all original contributions worked out in advance of the student's involvement. The student is being asked to execute a well-specified case study, to contribute to the methodology's operational artifacts, and to refine and extend the methodology through that execution. New methodological ideas, alternative directions, and substantive proposals from the student are welcome at any point — the document represents the PI's best current thinking, not a closed specification.

The expectation is high—this is a serious software-engineering project with a methodology paper, a software paper, and a reusable skill suite riding on it—but the work is well-defined and the support structure (weekly PI engagement, structured logs, the generality-testing discipline) is in place. With reasonable effort and honest communication, the student should succeed and emerge with substantial co-authorships, a major software accomplishment, and a clear contribution to a research program with longer-term funding trajectory.

The student should treat this document as the authoritative reference for the project's intent. Questions, ambiguities, or proposed deviations should be raised with the PI rather than resolved unilaterally.

End of document.